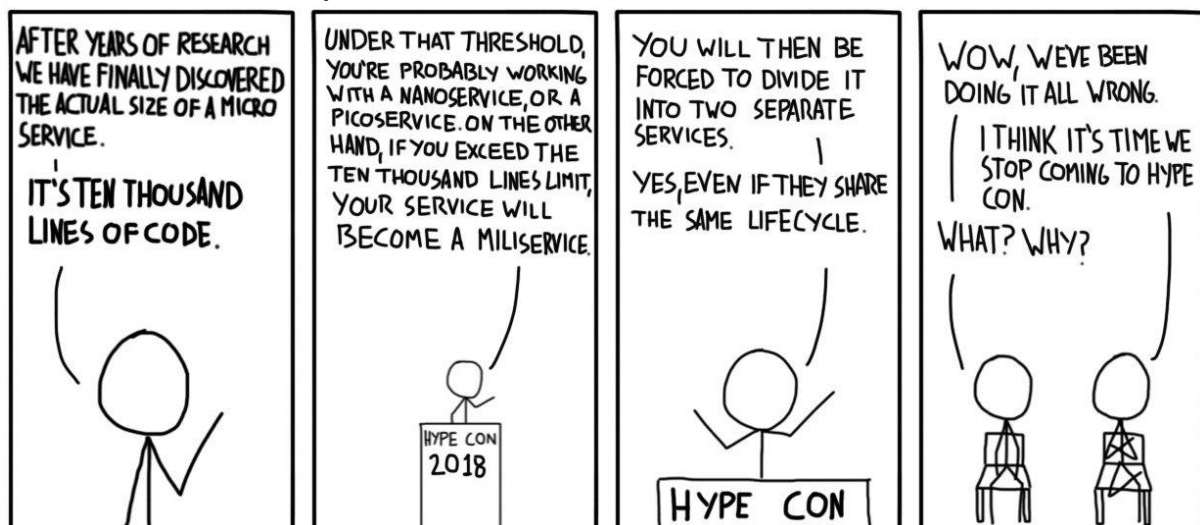


This exam project requires you to submit a report and your code on Toledo before June 15th, 9 AM.

Distributed Systems: Final project

Service-oriented distributed applications on a cloud platform



Master Electronics/ICT – Software Systems

Lab coaches:

- Bert Lagaisse
- Vivian Tsang
- Wout Tessens

1. Practical arrangements

The remaining weeks are allocated for the final exam project on cloud applications. You will complete this project with the same group of four students. **The deadline for submission on Toledo is June 15th, 9 AM CET.** Each student must submit the solution of the team individually on Toledo.

You will need to defend your solution with your full team during the exam period at the end of June (June 25th-27th). The defense will take place in the days after your written theory exam, such that you have processed the learning material of the lectures before the oral defense. This defense will consist of a 20-minute presentation and 10 minutes of questions. Make sure each team member takes an active role in the presentation. Some questions might ask for clarifications of your solution based on concepts from the lectures. However, the oral defense will not include exam questions about the concepts from the lectures.

This project is the basis for an oral exam for which your team must work independently to complete the following learning goals:

- Designing, developing, deploying and running a complex, distributed, multi-tiered and service-oriented cloud application;
- Learning how to implement a REST service or SOAP service from scratch;
- Learning how to implement a web application from scratch;
- Independently learn a complex middleware platform based on documentation on the web;
- Learning how to design a distributed system using different services of a PaaS platform such as authentication services, NoSQL services and message-oriented communication services;
- Learning to inspect and debug multi-tiered distributed applications.

The project is divided as follows:

- **Level 0: the bare minimum** to just pass the exam.
- **Level 1: the basic requirements** to pass the exam with a *sufficient* score, assuming you complete these requirements correctly and you can motivate your design choices. We expect each team to complete at least these requirements.
- **Level 2: the advanced requirements to pass the exam with a good to very good score:** you may decide yourself whether you want to complete these. Each requirement you implement will contribute to your score, based on the quality of your implementation.
- **Bonus level:** bonus requirements for students with lots of spare time.

This assignment must be carried out in groups of *four* people. Try to find teammates who will complete the same level of the project: either your team works together to complete the basic level, or your team works together to complete (some of) the advanced requirements. Your deliverables are:

- The code of your project, which implements all requirements up to the level that you choose to complete.
- A report in which you discuss the architecture and design decisions and show additional insights into deploying a cloud application. This report should answer the questions at the end of this assignment.

To submit the code and the report, you should create a zip file of your implementation. Please do not create a zip that packs your entire project folder with all libraries, as this would make your submission unnecessarily large. Just the maven build files and code that you created are sufficient.

2. Introduction

Main goal: In this project, you will learn designing, building, deploying and running a distributed, service-oriented web application. This distributed system is a composition of distributed, remote,

independent services of different companies. These services are running on different servers in different data centers and are managed in a different administration domain.

You can use the Spring Boot framework to accelerate the development of distributed enterprise-grade applications. The basic requirements are to develop a distributed application of which the distributed service-oriented architecture runs on minimal 3 VMs in 3 different Azure data centers.

Optionally, you can further support high availability and scalability requirements by deploying your application on the Microsoft Azure PaaS cloud platform, and leverage the paid and free services of the PaaS platform. There, you can also learn to work with the NoSQL storage services for persisting data scalable while maintaining consistency.

3. Assignment summary

You must design and develop a web shop that acts as a broker for multiple product suppliers.

The web shop of the broker must offer a *packaged, composite product or service* that requires external products or services from multiple suppliers, and that must be ordered as a whole atomically. For example, you could design and develop a travel agency that sells packaged travel arrangements, consisting of a hotel, rental car and flight. Or, you could design a meal broker that allows to order different meals from different restaurant chains, as well as a delivery service of your choice that will pick up the meals and deliver to your house. All of these examples have one property in common: there is *atomic all-or-nothing semantics* when the final order is placed on the broker's website to buy the composite, packaged service. This composite, packaged service must be executed by multiple external supplier companies with their own SOAP or REST service. Each part of the composite order will be processed at the side of the supplier. Hence, it requires distributed B2B (business-2-business) communication as well as a distributed atomic transaction.

The suppliers have their own business-2-business SOAP or REST service with a list of their products, and their real-time stock availability. The broker's web shop is not allowed to sell a product that is not in stock. Suppliers do provide their products to many other web shops too. Your broker web shop is not their only customer. For this assignment, you must also design, develop and host the REST or SOAP services of the suppliers on separate, external VMs in a datacenter of your choice (e.g. Azure VMs). Optionally, you can also deploy the services of the suppliers on other more lightweight compute services: e.g. Azure App service, containers or Azure Functions.

We leave it up to the imagination of the students to define their kind of web shop or broker of their choice. You can use any free or paid service from Azure, but it is more safe to stick to the services that provide free quota on a monthly or yearly basis. See the implementation hints further for more detail.

4. Requirements

We now outline the requirements that your implementation of the assignment must fulfill. These depend on the level that you choose to complete; as a reminder, only the basic requirements are what we expect from each team, while the advanced requirements are optional. The bonus requirements at the end are for the students that have plenty of spare time this semester.

1. Basic Requirements

You will design and build a fully functional distributed application that serves as the front-end (GUI) and back-end (application logic and data storage) of the broker platform. You will also design and develop at least two external supplier services that offer functionality to list, reserve and order products or services. You will ensure that the broker platform is fault tolerant even when relying on an external service. The broker should not crash because one of the suppliers is not responsive, or one of the suppliers returns a malformed response.

The order of the composite, packaged product must satisfy the ACID properties and must be implemented as a distributed transaction between broker and supplier services, with the broker as the transaction coordinator. The order will only succeed if:

- All items in the packaged product are available.
- The customer has provided a delivery address and valid payment information.

You should demonstrate that your solution works in different failure scenarios: one of the suppliers crashes, the network communication between the distributed servers is failing, or the broker crashed before the supplier could confirm the order. The ACID properties must be guaranteed in all these scenarios.

Users of your web shop should be able to order without creating an account or authenticating. Providing their delivery address and payment information at the end of the order process is sufficient. You should also not implement a real credit card payment protocol, and can of course just simulate this. *Managers* must be able to authenticate to see a list of all orders of all customers.

You will use one of the Azure data stores to persist and query the application data of the broker. You can choose between any SQL-based or NoSQL database that is available. For development and testing purposes, you could use a local database on your laptop, which you can then replace with the Azure storage solution once you deploy the broker to the Azure cloud.

For the remote supplier services, you should store the products and their stock in the appropriate data center of that supplier. Hence, compared to the broker, that is in a different data base in a different data center, managed by a different administration domain (e.g. a different Azure account of another team member). This will typically be the same data center as where you host the supplier service.

The three administration domains of the broker and at least two suppliers should be separated over three azure accounts of three students of the team. Each account then simulates one administration domain of one company involved in the service-oriented application.

2. Implementation hints

Through the Spring Boot middleware, you can quickly implement and launch a basic web application. You can also implement the REST or SOAP service of the suppliers using Spring Boot, similarly as we did in the earlier practical sessions. Spring has a good tutorial to get you started to create a REST service from scratch with the latest Spring Boot version:

- <https://docs.spring.io/spring-boot/tutorial/first-application/index.html>

To create a Spring Boot MVC web application, without authentication:

- <https://spring.io/guides/gs/serving-web-content>

To create a Spring Boot MVC web page with basic authentication, you can follow this tutorial:

- <https://spring.io/guides/gs/securing-web>

If you already have burned your Azure budget, do not worry. You actually don't need the Azure credits to pass this project. Azure offers services that include monthly free quota, which are more than sufficient for your project.

- Linux Virtual Machines, Azure App Service, Azure Functions, Azure Container Apps
- Azure Database for MySQL or PostgreSQL, Azure SQL Database, Azure Cosmos DB (NoSQL)

If you still have some Azure budget left, you can use paid services such as the very cheap table storage and persistent message queue. This can be needed for some of the advanced requirements.

3. Fall-back scenario to the bare minimum requirements (level 0)

If you are not able to complete the basic requirements due to other ongoing projects, a lack of time, or any other personal situation: the bare minimum to pass the exam project is a demonstration on your laptop of the working distributed application executing as three different processes on your laptop. This should include a Spring boot MVC web application as the broker, and two REST services as suppliers. These three processes should execute the distributed transaction for an order according to the ACID properties and should guarantee consistency of the data in case of a node failure or communication failure. At least one of these failure scenarios must be demonstrated. We also expect basic authentication of the manager.

4. Advanced requirements (Level 2)

Application-level access control. Users and managers should now authenticate using an external identity provider. Managers should be able to get an overview of all orders, and the state of the order (ongoing, succeeded, or failed). Other users should not be able to see this overview, even if they tamper with the URL. You will authenticate users and managers of the broker platform by implementing authentication and authorization through the free service **Auth0**. Auth0 is a typical example of Identity and Access Management as a Service. IAMaaS is a cloud-based solution that simplifies the management of digital identities and access to web applications (both authentication and authorization). Auth0 uses the concept of JWT tokens for authentication and authorization.

You can create a free account, without a credit card, via <https://auth0.com/signup>

Auth0 offers support for Spring Boot MVC, for which there are tutorials and sample code:

- <https://auth0.com/docs/quickstart/webapp/java-spring-boot>
- <https://github.com/auth0-samples/auth0-spring-boot-login-samples/tree/master/mvc-login>

Also think about how you could secure the interactions between the broker and the supplier services. How should the broker authenticate to the supplier service? With which identity? With which permissions? Design and implement an authentication and authorization approach for these interactions. Suppliers do not accept orders of just any HTTP client on the internet.

Loosely coupled communication between broker and suppliers. When one of the supplier services is not available or responsive when the customer wants to order, the broker will put the order in a message queue, and a background process will keep trying to complete the order for the next 15 minutes. The customer can leave the broker's web shop while the broker is trying to complete the order. After 15 minutes, the order should either succeed or fail, while respecting the ACID properties. Note that it is not necessary that all suppliers are responsive at the same time. The full order and protocol just need to complete within 15 minutes.

Lightweight hosting of services. At the basic requirements level, you can use standard IaaS compute services to execute the online services on Azure Virtual Machines. At this advanced level you can choose to deploy your application onto one of the more lightweight compute services (e.g. Azure Container Apps, or Azure App Services). A tutorial to deploy a full Spring Boot app with database on Azure App Service is available at: <https://learn.microsoft.com/en-us/training/modules/deploy-java-spring-boot-app-service-mysql/>

5. Bonus requirements (Bonus level)

- 1) All the different distributed services are implemented with Spring Boot if you stick to the basic requirements. To deal with the heterogeneity of a real service-oriented architecture, you could implement one of the supplier services in a different technology (e.g. PHP or Node.js).
- 2) Implement the web shop as a single page application instead of a regular web app.

5. Reporting

We would like you to report on your design and the decisions you have made (software architecture, system architecture, deployment diagram, design choices, possible alternatives and trade-offs), and on your understanding and implementation of distributed concepts.

Do not forget to put your names and team number in the report, and upload the report (as one PDF), together with your code (as one zip). For each team member, mention the role and tasks of the team member as well as the amount of hours that the team member contributed to the project. Be honest and fair. Compile a report that answers the following questions, for the levels that you have chosen to complete:

- Describe the distributed system architecture and deployment diagram of your solution. Include a brief summary of the different remote interfaces and their operations.
- Where and how did you enforce and implement access control such that only authorized users can access manager methods?
- Is there a scenario in which your implementation of ACID properties may lead to one of the properties not being achieved (atomicity, consistency, isolation, durability)?
- How does your application cope with failures of the external supplier services? How severely faulty may that service become before there is a significant impact on the functionality of your application?
- How have you structured your data model (i.e. the entities and their relationships), and why in this way?
- Compared to a relational database, what sort of query limitations have you faced when using a NoSQL database?
- What are the pitfalls when migrating a locally developed application to a real-world cloud platform? What are the restrictions of Azure in this regard?
- Provide a small cost analysis for your system architecture. What is the cost for each company in the service-oriented architecture.
- How extensive is the tie-in of your application with Azure? Which changes do you deem necessary for migrating to another cloud provider?
- Include the URL where your application is deployed. Provide a login and password for the teaching team and mention it in your report. Make sure that you keep your application deployed until after you have received your grades.

6. Submission

As described more elaborately in the practical arrangements, we expect from **each** student

- **One zip file** with the source code and maven build files. Don't include binaries or libraries.
- **One PDF** with the report and all diagrams in the PDF.

The deadline for submission on Toledo is June 15th, 9 AM CET

Good luck,

The Teaching Team.